



南開大學
Nankai University

人工智能技术实验 实验报告

实验名称：八皇后问题求解

姓 名： 殷家兴

学 号： 2313666

专 业： 自动化

人工智能学院

2025 年 10 月

一 问题简述

八皇后问题，是一个古老而著名的问题：如何能够在 8×8 的国际象棋棋盘上放置八个皇后，使得任何一个皇后都无法直接吃掉其他的皇后？为了达到此目的，任两个皇后都不能处于同一条横行、纵行或斜线上。

最早是由国际西洋棋棋手马克斯·贝瑟尔于 1848 年提出，之后陆续有数学家对其进行研究，其中包括高斯和康托。

八皇后问题可以推广为更一般的 N 皇后摆放问题：这时棋盘的大小变为 $N \times N$ ，而皇后个数也变成 N 。

二 实验目的

1. 通过求解皇后问题，熟悉深度优先搜索法技术；
2. 理解递归回溯算法思想，进而推广到 N 皇后问题；
3. 对实验进行图形化界面设计，实现按步或按解的展示。（选做）

三 实验内容

1. 实现八皇后问题的解法统计；
2. 将八皇后问题推广到 N 皇后；
3. 图形化界面设计。（选做）

四 实验步骤（包括重要程序代码解释）

八皇后问题的递归回溯算法思想十分朴素，当把问题分成若干步骤并递归求解时，如果当前步骤没有合法的选择时，则函数调用上一层的递归，此即为回溯。

每次的正向递归是一个试探的过程，将本次的影响带入到下一次的递归过程中去，如果在下一次的函数中，并没有找到一个合适的解去调用再下一层的函数，则证明上一轮的函数过程产生了错误的中间结果，该结果造成后续函数过程无法得到可行解，所以回溯返回至上一层递归，然后再次去试探，一直到达递归的出口（即可行解）结束本次试探。

八皇后问题的思路即：

- 以行的方式去摆放棋子；

- 在每次的试探摆放时，检查是否有以前的棋子与本次拟摆放的位置相冲突，（由于是按行摆放，故只需验证是否有同一列和同一对角线即可）；
- 如果产生了冲突，则返回上一层的调用；
- 如果产生了符合条件的位置，则进入下一行的判断。

依据该思路，编写的求解八皇后问题的 Python 代码如下：

```

1 class EightQueen:
2     def __init__(self):
3         # 皇后的数量
4         self.size = 8
5         # 创建一个包含8个-1的列表，表示棋盘，索引位表示行号，储存值
          表示列号
6         self.board = [-1] * self.size
7         # 用于存储所有解法
8         self.solutions = []
9
10        # 判断在指定行和列是否能够放置皇后
11        def is_safe(self, row, col): # row行，col列
12            for row_now in range(row):
13                if (self.board[row_now] == col or # 是否在同一列
14                    abs(self.board[row_now] - col) == abs(row_now - row)
15                    ):# 是否处于同一对角线，相距列、行数相同则判定为
                       是
16                return False
17            return True
18
19        # 递归解决八皇后问题，从指定行开始尝试放置皇后
20        def solve(self, row=0):
21            if row == self.size:# 当行数达到皇后数量停止递归
22                self.solutions.append(self.board[:])# 找到一种结果并添加
                    到solutions中
23                return
24            for col in range(self.size):
25                if self.is_safe(row, col):
26                    self.board[row] = col
27                    self.solve(row + 1)
28                    self.board[row] = -1# 实现回溯，当完成一种解法或此解
                        法不行时进行重置

```

```

29     # 返回所有找到的解法,用于外部访问
30     def get_solutions(self):
31         return self.solutions
32
33     # 打印结果
34     def print_solutions(self):
35         print(f"八皇后问题共有{len(self.solutions)}种解法")
36
37 if __name__ == '__main__':
38     eight_queen = EightQueen()
39     eight_queen.solve()
40     eight_queen.print_solutions()
41     # 打印详细结果
42     print("每种解法各行皇后的列坐标如下: ")
43     for solution in eight_queen.get_solutions():
44         print(solution)

```

这里设置皇后数量 `self.size` 为 8, 并根据皇后数量创建棋盘 `self.board` (同时完整填充的棋盘也可以表示一组可行解), 在代码运行过程中棋盘内容会不断回溯和更新。当找到一组可行解时, 此时的棋盘内容会存储到解空间 `self.solutions` 中并清空, 进行下一次试探。

`is_safe` 函数用于检验新放置的皇后是否与原有皇后冲突, `row` 表示行数、`col` 表示列数。函数中对某一位置加入新皇后是否会造成新皇后和原皇后处于同一列或同一对角线进行判断, 从而实现对皇后位置是否可行的检验。

`solve` 函数内部通过递归实现回溯。当棋盘 `self.board` 中没有填满时通过 `solve` 函数递归进行试探, 通过不断试探与回溯寻找可行解; 当完整找到一种可行解时, 棋盘 `self.board` 包含的信息就会写入解空间 `self.solution`, 然后继续寻找下一个可行解, 直至所有可行解都被找出停止。

对于 **N 皇后问题**, 对原八皇后问题代码进行改进, 增加读取皇后数量函数 `get_size` 实现:

```

1 # 获取皇后数量
2 def get_size(self,size=8):
3     if size < 4:
4         print("数据过小")
5         return False
6     elif size > 15:
7         print("数据过大, 算力不够! ")
8         return False
9     else:

```

```

10     self.size = size
11     # 创建一个包含size个-1的列表，表示棋盘，索引位表示行号，储存
      值表示列号
12     self.board = [-1] * self.size
13     return True

```

这里对皇后数量进行了限制，当皇后数量小于 1 时，不存在讨论意义；当皇后数量大于 15 时，可行解过于庞大导致求解时间过长。因此，这里将皇后数量限制在了 1 15 这一范围内。

对于 N 皇后问题, 我实现了**可视化**。通过导入 **pygame** 模块实现 N 皇后问题可视化, 可视化界面如下图所示。可视化界面实现了皇后数量的输入、解法显示时长、解空间显示等功能, z 最终实现效果可以直接通过与本报告同时提交的 **NQueen_可视化.exe** 看到。



图 1: 可视化界面

五 实验结果

根据 **NQueen.py** 求得不同皇后数量下的可行解数统计表如下:

表 1: 皇后数量与可行解数统计表

皇后数量	1	2	3	4	5	6	7	8
可行解数	1	0	0	2	10	4	40	92
皇后数量	9	10	11	12	13	14	15	—
可行解数	352	724	2680	14200	73712	365596	2279184	—

六 分析总结

八皇后问题使用的递归回溯算法，体现了深度优先搜索的思想，是深度优先搜索的一种实现方法。深度优先搜索强调的是尽可能深地搜索树的分支，直到达到目标节点或无法再深入为止，然后回溯到上一个节点继续搜索。

回溯算法每次的正向递归是一个试探的过程，将本次的影响带入到下一次的递归过程中去，如果在下一次的函数中，并没有找到一个合适的解去调用在下一层的函数，则证明上一次的函数过程产生了不正确的中间结果，这个结果使得整个函数不能被正确执行了，所以此时应回溯返回，然后再次去试探，一直到达递归的出口。

此外我还对 Python 语言的使用和 Python 构建可视化页面有了一定经验，初步学会使用 Python 语言编写程序以及利用 pygame 搭建基本的图形页面。